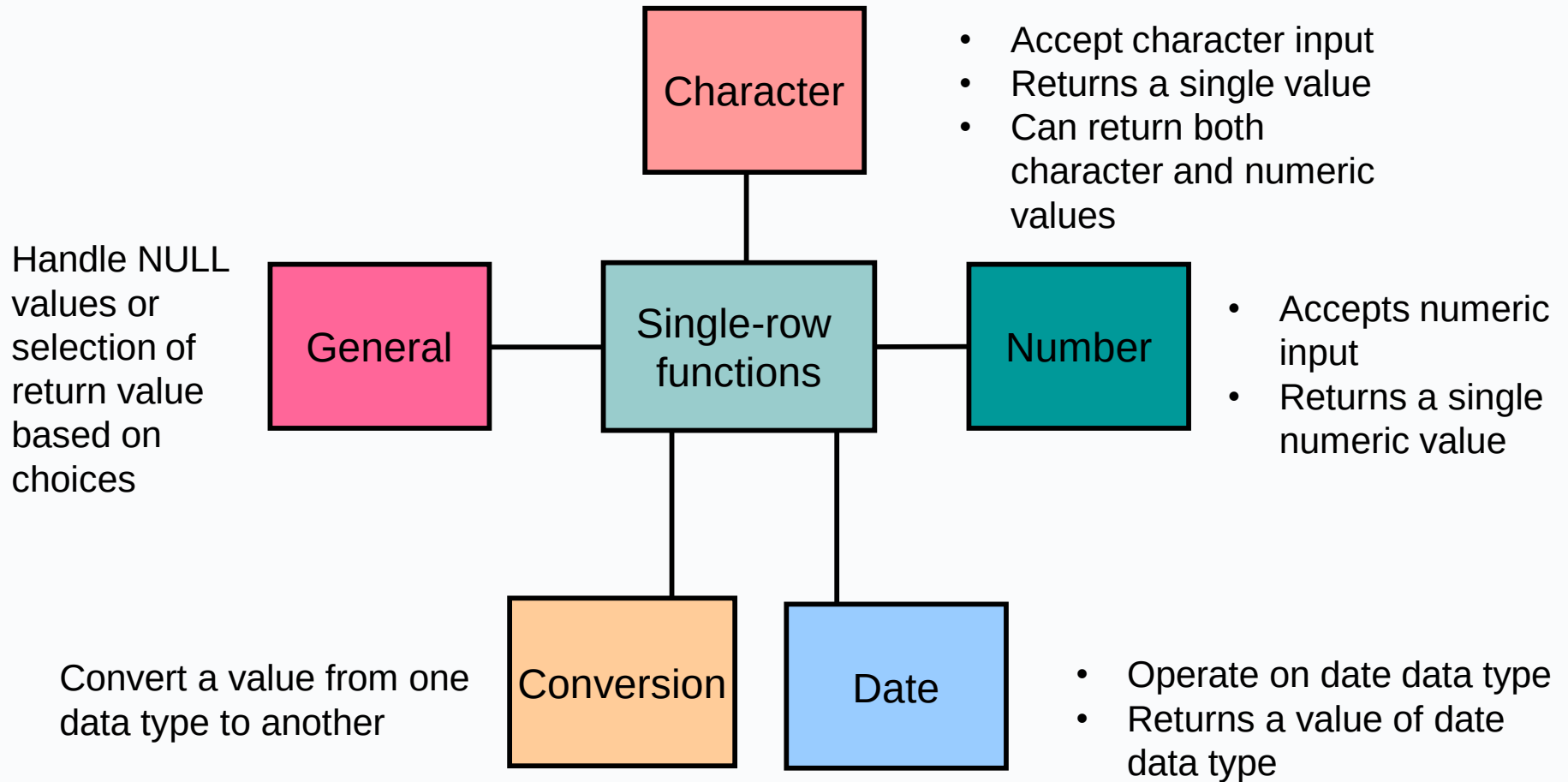


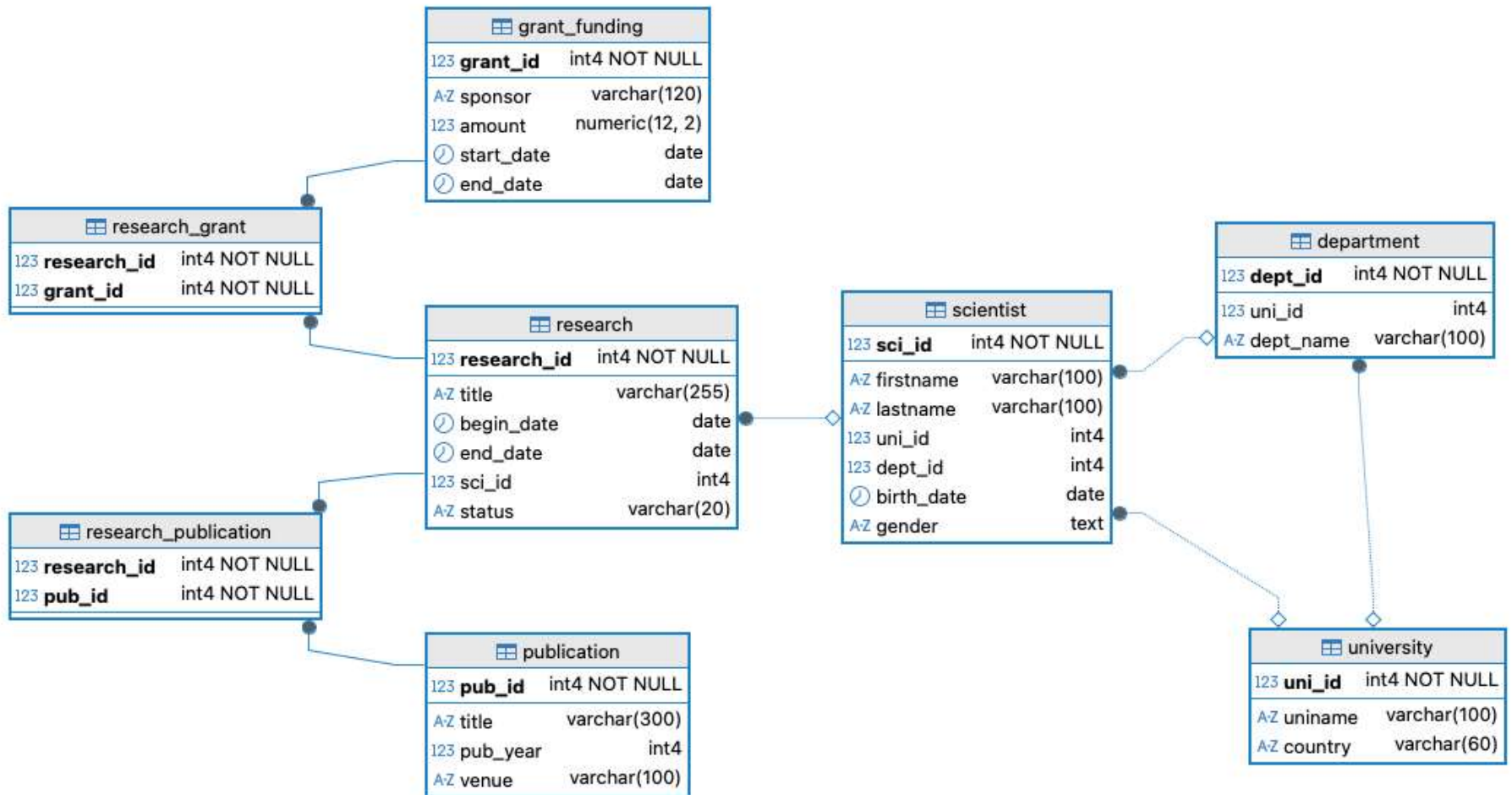
SQL Functions – Handling NULL values, numbers and dates

Dr Mariana Rocha
School of Computer Science

CPMU 2007 Databases 1, Lecture 5

Single-row functions





General functions

nullif(x,y)

- NULLIF is a accepts two arguments and returns a NULL if the two arguments are equal. Otherwise, it returns the first argument.
- It is useful when we need to replace specific values with NULL.
- Suppose we don't want the user to see SFI as one of the sponsors:

```
SELECT sponsor, nullif(sponsor, 'SFI')  
FROM grant_funding;
```

coalesce()

- Used to handle NULL values in a database query by returning the first non-NULL value from a list of expressions or column values.
- It provides a default or fallback value when NULL values are encountered in the data.
- Suppose you want to retrieve the scientist's date of birth and replace it with their firstname in case the value is null:

```
SELECT coalesce(birth_date::text,firstname)  
FROM scientist;
```

greatest()

- Returns the greatest value among a list of values.
- Suppose you want to know which one is the most recent: the project end_date or today's date.

```
SELECT research_id, title,  
GREATEST(end_date, CURRENT_DATE) "Most  
recent date" FROM research;
```

least()

- Opposite of greatest(): returns the smallest value among a list of values.
- Suppose you want to know which one is the oldest: the project end_date or today's date.

```
SELECT research_id, title, LEAST(end_date,  
CURRENT_DATE) "Most recent date" FROM  
research;
```


Number functions

Number Functions

- ROUND: Rounds value (up) to specified decimal
- TRUNC: Truncates value to specified decimal
- MOD: Returns the remainder of the division

Function	Result
ROUND (45 . 926 , 2)	45 . 93
TRUNC (45 . 926 , 2)	45 . 92
MOD (15 , 4)	3

Number Functions

--Round

```
SELECT grant_id, amount, ROUND(amount, 1) "Rounded  
amount" FROM grant_funding; --we could also do  
round(amount)for no decimal number
```

We can also use **ceil()** [round up to the nearest integer]
and **floor()** [round down to the nearest integer]

--Ceil, floor and round

```
SELECT grant_id, amount, ROUND(amount) "Round",  
ceil(amount) "Round up", floor(amount) "Round down" FROM  
grant_funding;
```

Using the ROUND Function

```
SELECT ROUND(450.9234), ROUND(450.9234,-  
1)/*nearest 10s*/, ROUND(450.9234,-2)/*nearest  
100s*/, ROUND(450.9234,-3)//*nearest 1000s*/;
```

- Negative second argument indicates rounding to the left of the decimal point
- Using -1 as the second argument in the function round() means you aim to round to the nearest 10s (up or down); using -2, to the nearest 100s; using -3, to the nearest 1000s etc.

Using the TRUNC Function

- Truncate works similarly, but without rounding the number.
- The decimal numbers are dropped, and the number on the left is truncated to 0 if the argument is -1, to 00 if the argument is -2, etc.
- With negative second argument, TRUNC will make value zero to left of decimal point, so 45 becomes 40

--Truncate

```
SELECT TRUNC(450.923, -  
1)/*truncate to nearest 10s*/,  
TRUNC(450.923,-2),  
TRUNC(450.923,-3) /*truncate to  
nearest 100s*/;
```

Using the TRUNC Function

- In finance, truncate() can be useful to discard insignificant decimal places to avoid rounding that could affect the balance, inflating or deflating amounts.
- We can also use it when a system operates on strict thresholds. Suppose an app operates with fixed resource limits on the database – truncating can ensure that the resource count remains within bounds without rounding up.

Mod

- MOD: Returns the remainder of the division.
- Used for detecting even/odd patterns
- Helps to categorise based on numerical patterns

Mod

- Suppose the university regularly reviews new research projects, and each project must be assigned to one of 3 review committees:

Committee ID	Description
0	Ethics & Compliance
1	Methodology Review
2	Funding Allocation

- Instead of manually distributing projects, you want a quick automatic rule that balances the load evenly among committees.
- We can use the mod of the `research_id` to allocate the projects to each committee.

Mod

Committee ID	Description
0	Ethics & Compliance
1	Methodology Review
2	Funding Allocation

```
SELECT
research_id,
title,
MOD(research_id, 3) "Assigned committee"
FROM research r
ORDER BY 3;
```

abs()

The absolute() function returns the absolute value of a number, even if it is a negative one.

For example, suppose we want to see if there is a difference between the end date of the research project and the end date of the funding.

We don't want any negative values, though.

--abs(): Suppose we want to know the difference in days between the end of the research and the end of grant. We do not want negative values, though

```
SELECT r.end_date "Research ends", g.end_date "Grant ends",  
abs(g.end_date - r.end_date) "Days variance" FROM research  
r JOIN research_grant rg USING (research_id) JOIN  
grant_funding g USING (grant_id);
```

Working with Dates - formatting

The function `to_char` can be used to display data in a certain format

```
SELECT birth_date, to_char(birth_date,  
'DD/MM/YYYY') FROM scientist;
```

Working with Dates – current time

The `now( )` function can retrieve the current date and time. It is given as a timestamp data type.

```
--current time
```

```
SELECT now();
```

```
SELECT current_date;
```

```
--current time data type
```

```
SELECT pg_typeof(now());
```

Arithmetic with Dates

- Adding seven (7) to a value stored in a date column produces a date that is one week later than the stored date.

```
--date in a week from now  
SELECT current_date + 7;
```

- To calculate the duration of a project in weeks, we can divide the number of days by 7.

```
--project duration in weeks  
SELECT (end_date - begin_date) / 7 from research;
```

Arithmetic with Dates

- We can also calculate the difference between the current date and a specific value using the function **age()**

```
--calculate scientist's age  
SELECT age(birth_date) FROM scientist;
```

```
--calculate projects's age  
SELECT age(begin_date) FROM research;
```

Conversion Functions

Function	Return Type	Description	Example
<code>to_char(timestamp, text)</code>	text	convert time stamp to string	<code>to_char(current_timestamp, 'HH12:MI:SS')</code>
<code>to_char(interval, text)</code>	text	convert interval to string	<code>to_char(interval '15h 2m 12s', 'HH24:MI:SS')</code>
<code>to_char(int, text)</code>	text	convert integer to string	<code>to_char(125, '999')</code>
<code>to_char(double precision, text)</code>	text	convert real/double precision to string	<code>to_char(125.8::real, '999D9')</code>
<code>to_char(numeric, text)</code>	text	convert numeric to string	<code>to_char(-125.8, '999D99S')</code>
<code>to_date(text, text)</code>	date	convert string to date	<code>to_date('05 Dec 2000', 'DD Mon YYYY')</code>
<code>to_timestamp(text, text)</code>	timestamp with time zone	convert string to time stamp	<code>to_timestamp('05 Dec 2000', 'DD Mon YYYY')</code>
<code>to_timestamp(double precision)</code>	timestamp with time zone	convert UNIX epoch to time stamp	<code>to_timestamp(200120400)</code>
<code>to_number(text, text)</code>	numeric	convert string to numeric	<code>to_number('12,454.8-', '99G999D9S')</code>

Cast()

- The function cast(attribute AS data type) is used to convert values into other data types for processing.
- The double colon syntax is a shorthand to cast values.

```
SELECT pg_typeof(cast(begin_date AS TEXT)) FROM  
research;
```

```
SELECT pg_typeof(amount::text) FROM grant_funding;
```


Using the TO_CHAR Function with Dates

- In our function, fm stands for **fill mode**.
- Fill mode removes any leading spaces or padding that would normally be added for certain format elements.

```
SELECT firstname, TO_CHAR(birth_date, 'fmDD  
Month YYYY') FROM scientist;
```

- In our formula, depth is used to format numbers as spelt-out ordinal words with appropriate suffixes (like "st", "nd", "rd", and "th").

```
SELECT firstname, TO_CHAR(birth_date, 'Month DDspth')  
FROM scientist;
```

Date Format of to_char

Format	Output
D	Day of week (1-7)
DD	Day of week (1-31)
DAY	Name of day right padded to display blanks up to length of longest day name
MM	Month (1-12)
MON	Abbreviated Month Name
MONTH	Full Month Name
YEAR	Year spelt out
YYYY	4 digit year
YY	2 digit year

Suffixes for numbers:

TH e.g DDTH would be perhaps 4TH

SP spelled number DDSP FOUR

SPTH spelled number plus TH FOURTH

Using the TO_CHAR Function with Numbers

```
TO_CHAR(number, 'format_model') ddspth
```

These are some of the format elements that you can use with the TO_CHAR function to display a number value as a character:

Element	Result
9	Represents a number
0	Forces a zero to be displayed
\$	Places a floating dollar sign
£	Local currency sign
.	Prints a decimal point
,	Prints a comma as thousands indicator

Using the TO_CHAR Function with Numbers

--currency sign

```
SELECT to_char(amount, '€999,999.99') FROM  
grant_funding;
```

Create your own function Procedure language SQL (PL/pgSQL)

- PL/pgSQL (Procedural Language/PostgreSQL) functions allow you to write reusable, modular blocks of code that can perform complex operations not easily achieved with a simple query.
- PL/pgSQL functions extend SQL by allowing control structures such as loops and conditional statements.
- PL/pgSQL functions are often used as triggers, where a function is executed automatically in response to certain events on a table (like INSERT, UPDATE, or DELETE).

Syntax

```
CREATE OR REPLACE FUNCTION  
function_name(parameter_name data_type)  
RETURNS return_type AS $$  
BEGIN  
RETURN something;  
END;  
$$ LANGUAGE plpgsql;
```

```
--PL/SQL
```

```
CREATE OR REPLACE FUNCTION get_project_status(begin_d DATE,  
end_d DATE)  
RETURNS TEXT AS $$  
BEGIN  
IF current_date BETWEEN begin_d AND end_d THEN  
RETURN 'Active';  
ELSEIF current_date < begin_d THEN  
RETURN 'Upcoming';  
ELSE  
RETURN 'Completed';  
END IF;  
END;  
$$ LANGUAGE plpgsql;
```

```
SELECT research_id, title,  
get_project_status(begin_date, end_date) AS project_status  
FROM research  
LIMIT 5;
```